

Experiment- 5

Branch: MCA (AI&ML)	Semester: 2
Student Name: Ajay Rakwal	UID: 25MCI10329
Subject Name: Technical Training	Subject Code: 25CAP-652
Section/Group: MAM-1(A)	Date of Performance: 24-02-2026

Aim of the program:

To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic.

Software Requirements

- **Operating System:** Windows / Linux
- **Database Management System:** MySQL / Oracle / PostgreSQL
- **SQL Interface:** MySQL Workbench / Web Based / pgAdmin

Objective

- **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
- **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
- **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.

- Exception Handling: To handle cursor-related errors and performance considerations during large-scale data iteration.

Procedure of the Practical

- I. Open the database environment such as Oracle / MS SQL Server / PostgreSQL and connect to the desired database.
- II. Create a sample Employee table and insert records to simulate real-world row-wise data processing.
- III. Declare a simple forward-only cursor to fetch employee records sequentially and display them one by one.
- IV. Open the cursor and use the FETCH operation to retrieve each row for processing.
- V. Implement row-by-row manipulation logic such as updating salaries based on specific conditions.
- VI. Handle cursor execution flow by checking for end-of-data or empty result sets to avoid runtime errors
- VII. Properly close and deallocate the cursor after completing all iterations to release system resources.
- VIII. Execute the cursor-based blocks and observe sequential processing of records and updates performed.
- IX. Verify the final results using SELECT queries and analyze how cursors enable procedural control over row-wise operations.

Practical / Experiment Steps

--table creation

```
CREATE TABLE Employee (  
    emp_id INT,  
    emp_name VARCHAR(50),  
    salary INT );
```

--values insertion

```
INSERT INTO Employee VALUES (1,'Ajay',20000);
```

```
INSERT INTO Employee VALUES (2,'Jatin',15000);
```

```
INSERT INTO Employee VALUES (3,'Nidhi',25000);INSERT INTO Employee VALUES  
(4,'yuvraj',0);
```

```
Select * From Employee
```

--Exception handling performed in multiple lines

```
BEGIN
```

```
INSERT INTO Employee VALUES (4,'Anay',20000);
```

```
IF SQL%FOUND THEN
```

```
DBMS_OUTPUT.PUT_LINE('ROW INSERT SUCCESSFULLY');
```

```
END IF ;
```

```
DELETE FROM Employee WHERE emp_id =99;
```

```
IF SQL%FOUND THEN
```

```
DBMS_OUTPUT.PUT_LINE('ROW DELETE SUCCESSFULLY');
```

```
ELSE
```

```
DBMS_OUTPUT.PUT_LINE('emp_id NOT FOUND');
```

```
END IF ;
```

```
END;
```

```
Select * From Employee;
```

-- STEP 1 Forward only cursor

```
DECLARE
```

```
CURSOR emp_cursor IS SELECT EMP_ID,SALARY FROM EMPLOYEE;
```

```
V_EMP_ID Employee.EMP_ID%TYPE;

V_SALARY EMPLOYEE.SALARY%TYPE;

SALARY_ZERO EXCEPTION;

BEGIN

    OPEN emp_cursor;

    FETCH emp_cursor INTO V_EMP_ID ,V_SALARY;

--STEP 2 Row-by-Row Manipulation Logic

    WHILE emp_cursor%FOUND

LOOP

    IF V_SALARY=0 THEN

        RAISE SALARY_ZERO;

    END IF

    UPDATE EMPLOYEE

    SET SALARY=V_SALARY*1.10

    WHERE EMP_ID=V_EMP_ID;

    FETCH emp_cursor INTO V_EMP_ID ,V_SALARY;

    END LOOP

CLOSE emp_cursor;

--STEP 3 exception implemented

EXCEPTION

    WHEN SALARY_ZERO THEN

        DBMS_OUTPUT.PUT_LINE('Salary is zero. Increment not possible.');
```

END;

I/O Analysis (Input / Output)

Input

- Employee table creation queries
- Sample data insertion commands
- Cursor declaration and opening statements
- FETCH operations for row-by-row processing
- Conditional UPDATE logic inside cursor loop
- CLOSE and DEALLOCATE cursor commands

Output

- Records fetched sequentially using cursor
- Row-wise salary updates performed correctly
- Cursor lifecycle executed (declare, open, fetch, close, deallocate)
- Proper handling of empty or termination conditions
- Updated records displayed accurately after cursor execution

OUTPUT:

Table Created:

```
Data Output Messages Notifications
CREATE TABLE

Query returned successfully in 70 msec.
```

Insert sample records:

	emp_id	emp_name	salary
	integer	character varying (50)	integer
1	1	Ajay	20000
2	2	Jatin	15000
3	3	Nidhi	25000
4	4	yuvraj	0

--insertion of new entry with the use of exception handling

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	emp_id integer 🔒	emp_name character varying (50) 🔒	salary integer 🔒
1	1	Ajay	20000
2	2	Jatin	15000
3	3	Nidhi	25000
4	4	yuvraj	0
5	5	Anay	20000

--Implementing cursor lifecycle and exception handling as salary of one entry = 0

Data Output	Messages	Notifications
	<pre>NOTICE: Salary is zero. Increment not possible. DO Query returned successfully in 76 msec.</pre>	

Learning Outcome:

- Understand the concept and need of cursors for row-by-row data processing in databases.
- Learn the lifecycle of cursors including declare, open, fetch, close, and deallocate operations.
- Gain practical knowledge of applying procedural logic to individual records using cursors.
- Develop the ability to handle cursor-related exceptions and manage system resources efficiently.
- Apply cursor-based logic to real-world enterprise scenarios such as payroll updates and data migration tasks.